

Passo a passo para testar a concorrência

Prática de como simular e detectar conflitos de concorrência em PostgreSQL usando o nível de isolamento **SERIALIZABLE**



Etapa 1

1 Estrutura da tabela

Criamos uma tabela simples para simular o estoque de um produto:

```
CREATE TABLE produto (  
  id_produto SERIAL PRIMARY KEY,  
  nome VARCHAR(50),  
  estoque INT NOT NULL  
);  
  
INSERT INTO produto (nome, estoque)  
VALUES ('Notebook Gamer', 1);
```

Etapa 2

2 Criação da Procedure (com atraso)

A ideia é as duas transações lerem o mesmo estoque antes de atualizar, criando conflito sob SERIALIZABLE.

```
CREATE OR REPLACE PROCEDURE realizar_compra_demo(p_id_produto INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_estoque INT;
BEGIN
    -- 1) Lê o estoque SEM bloquear a linha
    SELECT estoque INTO v_estoque
    FROM produto
    WHERE id_produto = p_id_produto;

    -- 2) Simula processamento para aumentar a chance de corrida
    PERFORM pg_sleep(3);

    -- 3) Se havia estoque quando leu, tenta debitar agora
    IF v_estoque > 0 THEN
        UPDATE produto
        SET estoque = estoque - 1
        WHERE id_produto = p_id_produto;
        RAISE NOTICE 'Compra realizada!';
    ELSE
        RAISE NOTICE 'Produto esgotado!';
    END IF;
END;
$;
```

Etapa 3

3) Em duas janelas (Sessão 1 e Sessão 2) rode quase ao mesmo tempo:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
CALL realizar_compra_demo(1);  
COMMIT;
```

Resultado esperado: uma sessão confirma; a outra falha com ERROR: could not serialize access due to concurrent update.

 **Dica:** antes de repetir, reponha o estoque:

```
UPDATE produto  
SET estoque = 1  
WHERE id_produto = 1;
```

Verificando se deu certo

```
SELECT * FROM produto;
```

 O campo `estoque` deve mostrar novamente o valor 1.

Etapa 4



O que o `pg_sleep(3)` faz

Pausa a execução

`pg_sleep(3)` pausa a execução da transação por **3 segundos**.

Mantém transação aberta

Durante esse tempo, o banco **mantém a transação aberta** e **não confirma o COMMIT**.

Cria concorrência

Assim, outra transação que iniciar nesse intervalo **também lê o estoque antigo (1)** e tenta comprar.

Detecta conflito

Quando ambas tentam atualizar o mesmo registro, o PostgreSQL detecta um **conflito de serialização (erro 40001)**.

Resumo:

"O `pg_sleep(3)` faz o banco 'dormir' por 3 segundos, simulando um sistema real que está processando um pedido. Esse tempo é o suficiente para outra transação entrar em cena e causar a concorrência — exatamente o que queremos mostrar na prática."

